



Phase 1: Pre-Coding Design (The "OoO Hardware Contract")

Before writing code, you must align on these core out-of-order data structures and mechanics so your individual modules connect seamlessly.

- **OoO Hardware Structures:** Complete the definitions for `ROBEntry` and `RSEntry` in `Basics.h`. You must agree on the exact fields required (e.g., Valid, Ready, Architectural Register ID, computed value, exception flags).
- **The "Tick" Mechanism:** Establish the reverse-order evaluation loop in `Processor::step()`: Commit -> Execute & Broadcast -> Decode -> Fetch -> Latch Update. This simulates a single clock cycle and manages the Common Data Bus (CDB) safely.
- **Execution Unit Rules:** Agree on how the `ExecutionUnit` and `LoadStoreQueue` classes will simulate being pipelined internally. The LSQ is unique and must ONLY execute instructions in strictly sequential order.
- **Memory & ARF:** `Memory` is accessed sequentially by index (+1 per item, not +4), and starts from index 0. `ARF` strictly enforces `x0 == 0`.

Phase 2: Core Implementation Tasks

These are the discrete functional blocks that need to be built.

Front-End & System State

- **Assembly Preprocessor:** Write the `make run` target logic to parse the text file. Strip comments (`#`) and empty lines. Map memory labels (e.g., `.A: 1 2 3`) to sequential vector indices starting at 0, ensuring unpopulated sections are zeroed out.
- **Fetch Stage:** Fetch the instruction and determine the next PC using the `BranchPredictor`.
- **Decode Stage:** Allocate an entry in the ROB for *every* instruction. Rename registers using the RAT, allocate a Reservation Station, and dispatch. If the ROB or target RS is full, the instruction must stall.

Back-End & Out-of-Order Execution

- **Execute & Broadcast Stage:** Implement the logic for the Adder, Multiplier, Divider, Branch, Logic, and LSQ units.
 - At the start of a cycle, a unit executes the oldest ready instruction in its RS.

- When complete, broadcast the result on the CDB at the end of the cycle to update waiting RS entries and the ROB, then immediately deallocate the RS entry.
- **Exception Detection:** During Execution, detect exceptions (Math result exceeds `2**31-1` or `< -2**31`, division/remainder by 0, out-of-bounds memory access). Flag the exception in the broadcast, but **do not** update the main `Processor::exception` bit yet.

In-Order Commit

- **Commit Stage:** This is the ONLY stage that can alter architectural state. Look at the head of the ROB.
 - **Standard Commit:** If complete and exception-free, update the ARF or Memory.
 - **Branch Resolution:** Update the 2-bit saturating counter (States 0-3). If mispredicted, flush the entire pipeline and set the PC to the correct next instruction.
 - **Exception Handling:** If the ROB head has an exception flag, set `Processor::exception = true`, set the PC to the faulting instruction, flush the pipeline, and halt execution.

Phase 3: Integration & Testing

- **Makefile Setup:** Ensure `make compile FILE=<filename.cpp>` strictly produces the `main` executable, and `make run FILE=<filename.s>` successfully executes your preprocessor script.
- **Simulation Halt:** Ensure `Processor::step()` correctly returns `false` only when the ROB is completely empty and no more instructions are left to fetch.